

Разработка информационной системы для автоматизации автосервиса

Python + Flask + SQLite. Сквозной учет клиента, автомобиля, заказ-наряда, работ и запчастей.

Клиенты
Автомобили
Заказы

Работы
Нормо-часы
Стоимость

Склад
Резерв
Списание

Ручной учет разрывает процесс между клиентом, ремонтом и складом.

Данные дублируются

клиенты, автомобили и состав работ переносятся между журналами и таблицами

Статусы неуправляемы

заказ может зависнуть между открытием, ожиданием запчастей и выполнением

Склад не связан с ремонтом

резерв и фактическое списание материалов фиксируются несогласованно

Документы собираются вручную

заказ-наряд и итоговая стоимость требуют повторного пересчета

Вывод: системе нужен единый контур данных от обращения клиента до закрытия заказ-наряда.

Проект закрывает полный жизненный цикл заказ-наряда.

Цель

Разработать информационную систему, которая объединяет учет клиентов, автомобилей, работ, запчастей и документов автосервиса.

1. описать предметную область и требования

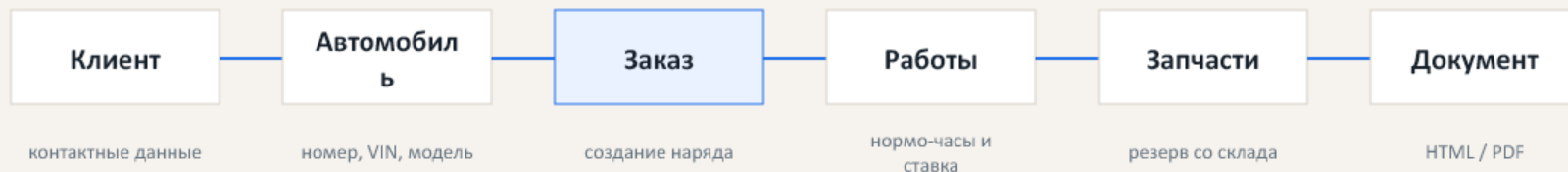
2. спроектировать модель данных

3. реализовать REST API на Flask

4. добавить правила статусов и склада

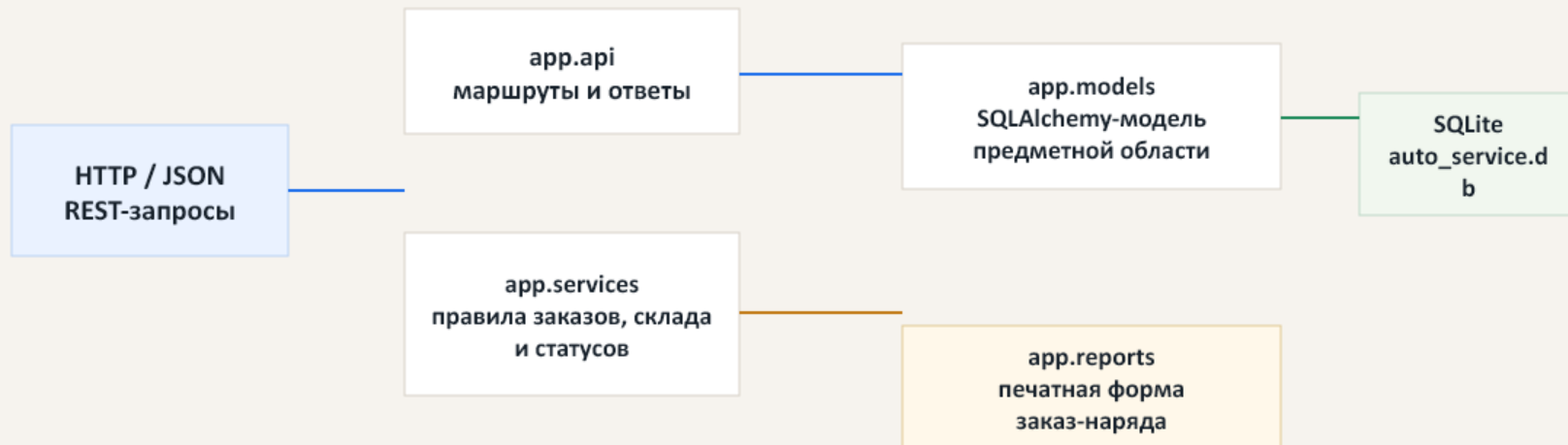
5. сформировать печатную форму заказ-наряда

Заказ-наряд проходит контролируемую цепочку от регистрации до закрытия.



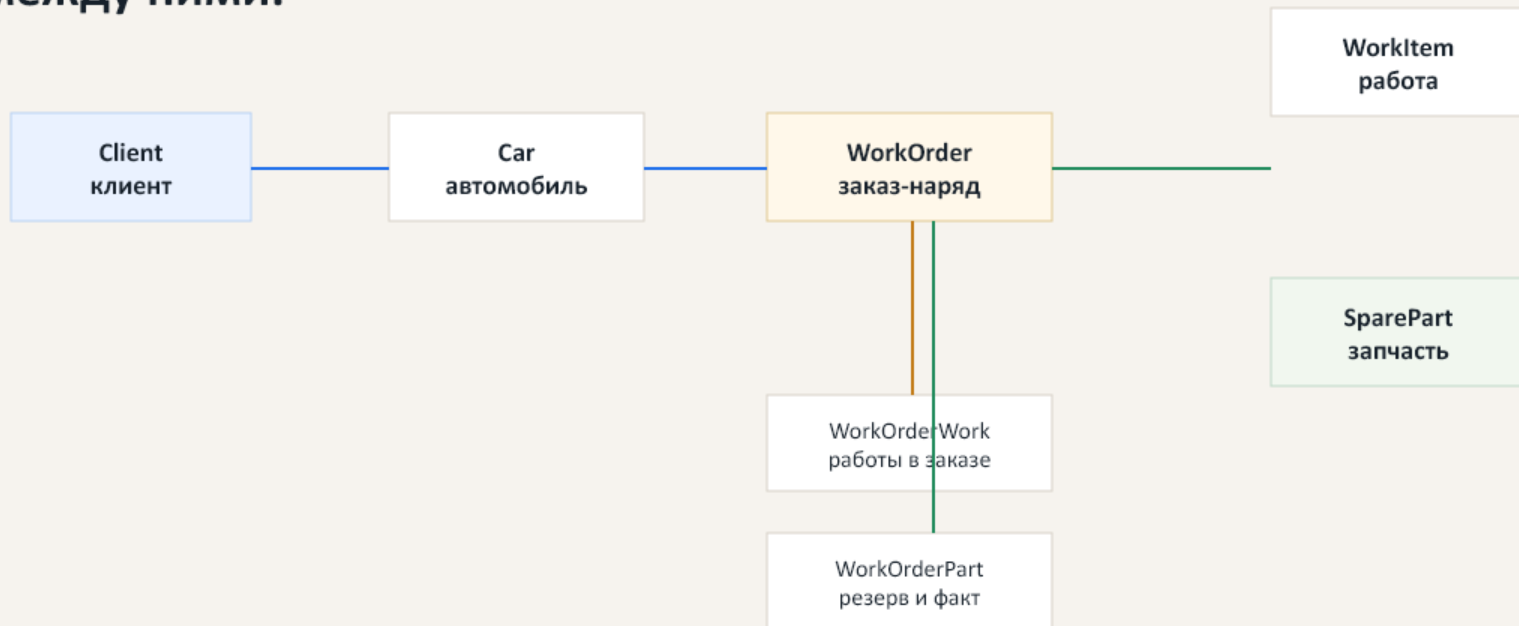
Ключевая идея реализации: каждое действие связано с заказом, поэтому стоимость, статус и складской остаток пересчитываются по единым правилам.

Flask-приложение разделено на API, бизнес-логику, модели и отчеты.



Технологии: Flask, Flask-CORS, Flask-SQLAlchemy, Pydantic, SQLite.

Сущности отражают реальные объекты автосервиса и связи между ними.



Связи многие-ко-многим вынесены в промежуточные таблицы: заказ хранит количество работ, резерв запчастей и фактическое списание.

Маршруты сгруппированы по справочникам, заказам и документам.

Справочники

GET/POST /api/clients
GET/POST /api/cars
GET/POST /api/works
GET/POST /api/parts

Заказ-наряды

GET/POST /api/orders
GET /api/orders/<id>
PUT /api/orders/<id>/status

Операции

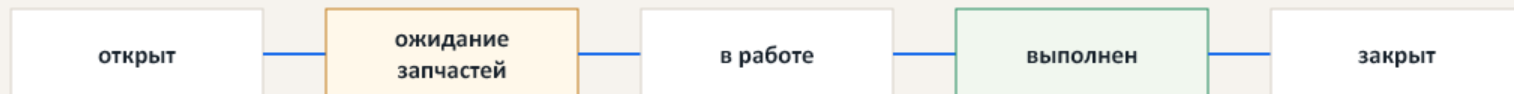
POST /add_work
POST /add_part
POST /use_part

Документы

GET /document?format=html
GET /document?format=pdf

API возвращает JSON и проверяет базовую целостность: клиент существует, автомобиль принадлежит клиенту, статус допустим, запчастей хватает на резерв.

Статусы и складские операции защищают заказ от некорректных переходов.



Резервирование
проверяет свободный остаток и
переводит открытый заказ в ожидание
запчастей

Списание
уменьшает склад и включает
фактически использованные запчасти
в стоимость

Завершение
автоматически списывает оставшийся
резерв перед статусом «выполнен»

Стоимость заказа пересчитывается из работ и фактически использованных запчастей.

SQLAlchemy-модель фиксирует связи заказа с клиентом, автомобилем, работами и запчастями.

app/models.py

```
class WorkOrder(db.Model):
    __tablename__ = 'work_orders'
    id = db.Column(db.Integer, primary_key=True)
    client_id = db.Column(db.Integer, db.ForeignKey('clients.id'), nullable=False)
    car_id = db.Column(db.Integer, db.ForeignKey('cars.id'), nullable=False)
    created_at = db.Column(db.DateTime, default=datetime.utcnow)
    status = db.Column(db.Enum(OrderStatusEnum), default=OrderStatusEnum.OPEN)
    total_cost = db.Column(db.Float, default=0.0)

    car = db.relationship('Car', backref='orders', lazy=True)
    works = db.relationship('WorkOrderWork', backref='order',
                           cascade='all, delete-orphan')
    parts = db.relationship('WorkOrderPart', backref='order',
                           cascade='all, delete-orphan')
```

Что важно показать на защите

- заказ связан с клиентом и автомобилем
- работы и запчасти вынесены в отдельные таблицы
- статус хранится как перечисление
- итоговая стоимость хранится в заказе и пересчитывается сервисом

Маршрут создания заказ-наряда проверяет целостность данных до записи в базу.

app/api.py

```
@app.route('/api/orders', methods=['POST'])
def create_order():
    data = request.json
    if not Client.query.get(data['client_id']):
        return jsonify({'error': 'Клиент не найден'}), 400

    car = Car.query.get(data['car_id'])
    if not car:
        return jsonify({'error': 'Автомобиль не найден'}), 400
    if car.client_id != data['client_id']:
        return jsonify({'error': 'Автомобиль не принадлежит клиенту'}), 400

    order = WorkOrder(client_id=data['client_id'], car_id=data['car_id'])
    db.session.add(order)
    db.session.commit()
    return jsonify({'id': order.id}), 201
```

Проверки перед созданием

1. клиент существует
2. автомобиль существует
3. автомобиль принадлежит выбранному клиенту
4. заказ создается только после успешных проверок

Резервирование запчастей учитывает уже занятый остаток и меняет статус заказа.

app/services.py

```
def add_part_to_order(order_id: int, part_id: int, quantity: float):
    if quantity <= 0:
        raise ValueError("Количество должно быть больше нуля")

    order = WorkOrder.query.get(order_id)
    part = SparePart.query.get(part_id)
    if not order or not part:
        raise ValueError("Заказ или запчасть не найдены")

    reserved_total = db.session.query(db.func.sum(
        WorkOrderPart.quantity_reserved
    )).filter(
        WorkOrderPart.part_id == part_id,
        WorkOrderPart.order_id != order_id
    ).scalar() or 0

    available = part.quantity_in_stock - reserved_total
    if available < quantity:
        raise ValueError(f"Недостаточно запчастей {part.name}")

    if order.status == OrderStatusEnum.OPEN:
        order.status = OrderStatusEnum.WAITING_PARTS
```

Смысл фрагмента

Код показывает не просто запись строки в таблицу, а бизнес-правило: система не даст зарезервировать больше доступного остатка и автоматически отражает ожидание запчастей в статусе заказа.

Создан рабочий прототип системы автоматизации автосервиса.

Единая база

клиенты, автомобили, работы, запчасти и заказ-наряды хранятся в SQLite

Контроль процесса

статусы заказа меняются по правилам, а не произвольно

Складской учет

система различает резерв и фактическое использование запчастей

Документооборот

формируется печатный заказ-наряд в HTML и простой PDF

Перспективы: роли пользователей, интерфейс оператора, журнал аудита, интеграция с учетными системами и расширенная аналитика загрузки.